

IoT Progress Report

IoT Connected SmartHub Dashboard

Thiago Costa, Johannes Hirt, and Anar Amartuvshin

Introduction

For our project, we are developing an IoT based SmartHub Dashboard using two ESP32 devices and multiple communication technologies. The system is designed for student use and is able to display a few necessary information such as time, music playback, and room temperature data. Bluetooth Low Energy (BLE) is implemented for short-distance communication between nearby devices, while LoRa is implemented for long-distance data transmission. WiFi is used for features such as web server access and remote interaction. The main ESP32 device acts as the central SmartHub, connecting to multiple components including the display, which contains its own ESP32 microcontroller, a secondary ESP32 LoRa node, a speaker, and other components. Additional components such as a camera and a servo motor are for auto rotating screens using a hand gesture. We are training our own model by taking many samples through the camera.

Overall, the goal of this project is to create an user-friendly dashboard that has access to multiple functionalities into a single smartHub device, providing students with convenient access to useful information. Everything from one desk smart hub.

Current Status

Overall:

After ordering the necessary components for our system, we were finally able to test out individual components and set up each wireless communication method. Also, we have updated our architecture as shown below in **figure 1**, where it shows the main controller to all subsystems such as display, LoRa transmitter, camera, BLE connection, webserver and servo. PlatformIO is used for a development environment for our coding.

- LoRa: (Johannes)

So far, we have successfully implemented the LoRa communication between two ESP32 Lora Dev boards where one node collects sensor data and transmits it back to the main smart Hub. Since the Heltec board contained the LoRa radio, we are able to establish a communication between the transmitter and receiver using the provided antenna. The system operates at the RF frequency of 915 MHz, which is in the US band as mentioned during the class. We have runned a test for 2 meter and 5 meter for long distance and confirmed successful data transmission between nodes. Moreover, the code is implemented based on the Arduino LoRa communication example code and has been modified for our system. We are also reading the Received Strength Indicator or RSSI for measuring how strong the received signal is in dBm and SNR or Signal to Noise Ratio for

checking how the signal is reliable. That way we can verify the LoRa communication and be able to check in 2m and 5 m in free space. Testing is currently in progress and we will upload more testing in our next report.

- Motor: (Johannes)

The system uses a high-torque servo motor to rotate the display based on user input. It is able to lift about 20kg. The servo is controlled by the ESP32 and enables automatic screen orientation through hand gesture commands, such as swipe in up, down, right and left. Tests completed so far include rotating the servo to specific angles based on user input.

Currently, the servo requires a 6–8 V power supply, and we have not yet identified a suitable battery. For testing purposes, we have been using a classroom power supply. The goal is to attach the servo to the display and achieve rotation of ± 90 degrees.

- Screen: (Thiago)

Moreover, the 7-inch CrowPanel display we purchased has been tested, and we have implemented a basic UI that includes features such as a timer and animated music playback bars, as shown in Figure 1. The screen is configured as an 800x480 RGB display. We are also using the open-source library LVGL for our GUI, and it draws the screen in chunks.

The screen is capable of rotating about both the portrait and landscape. In addition, we have successfully implemented Spotify music playback using a Spotify Premium account and we are now able to control play, stop, hit next on our spotify account. Also using the weather API, we are able to display the weather on the screen. The display has a couple pages the user can switch to using a side button.

We are also able to control the screen brightness.

- BLE and Wifi Connection: (Anar)

We aimed to implement a battery level indicator on the UI using Bluetooth Low Energy (BLE) communication. The initial idea was to retrieve battery status from nearby devices such as phones, earbuds, and tablets. However, Apple devices have strict security restrictions, and the ESP32 was unable to establish a connection, even though the devices were visible in the nRF Connect application. It appears that Apple devices do not advertise accessible BLE services for battery data to external devices like the ESP32.

As an alternative, we used Apple Shortcuts combined with a WiFi connection. This approach allowed us to successfully retrieve battery level information from the iPhone, as well as upcoming calendar events and reminders. How does the apple shortcut work?

First we request the data we want to retrieve and build them as blocks in the app. Then using webserver over wifi, it transmits data to the webserver and esp32 reads the value and displays on the OLED.

In addition, BLE communication has been successfully implemented for short-range use. We were able to establish connections with BLE helper applications such as nRF Connect, confirming that data can be transmitted and received correctly. Next step, we plan to detect nearby Android devices using their MAC addresses for identification purposes. Since Apple devices are not that accessible, we are aiming for android devices.

Work Completed

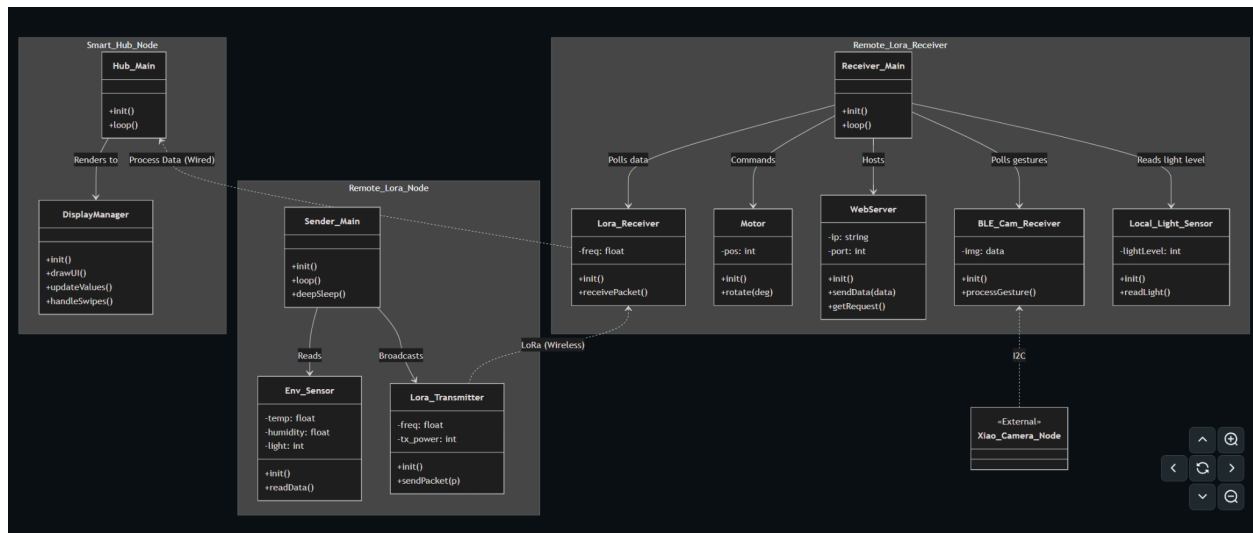


Figure 1: Current Architecture Diagram (Johannes)



Image 1: The example UI for the screen. (Thiago)

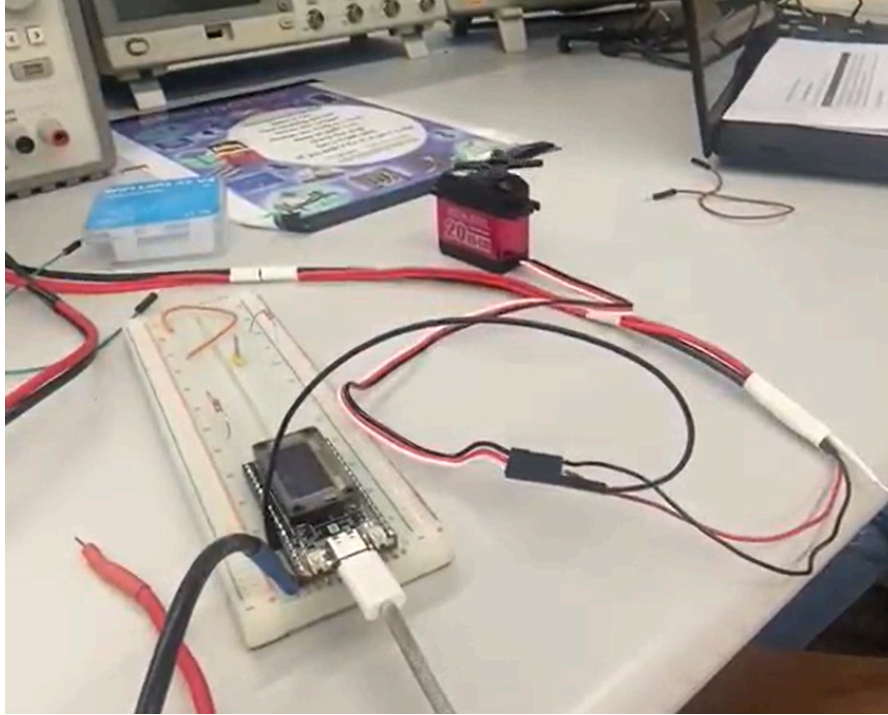


Image 4: Servo Set up. To watch a video: [Click here to watch a demo video](#) (Johannes)

Current Work

Camera Set up: We are currently working on implementing hand gesture recognition. The camera is being trained to detect gestures such as swipe right, swipe left, swipe up, swipe down, and idle states. So far, we have collected approximately 2000 samples for training. The next step is to label the dataset by annotating bounding boxes around the hand in each image. More information is shown in the **image 5** below. Since the ESP32 has very limited memory and progressing power, it can't handle large high-resolution color images. That is why images need to be sized in 96x96 pixels and converted to grayscale which reduces memory usage and computation.

```
Copied 2394 files to C:\IoT Data
Skipped (non-jpg): 0
up      : 294
down    : 240
lswipe  : 240
rswipe  : 225
rotate  : 225
idle    : 1170
```

Image 5: Camera gesture training progress.

Currently, the accuracy of the camera-based gesture recognition is relatively low due to model optimization techniques used to reduce latency and memory usage, making it suitable for deployment on the ESP32, as shown in **image 6**.

Model optimizations and performance

Model optimizations can increase on-device performance but may reduce accuracy. Performance estimate for Espressif ESP-EYE (ESP32 240MHz) - [Change target](#)

Quantized (int8)

Selected ✓

	IMAGE	CLASSIFIER	TOTAL
LATENCY	15 ms.	732 ms.	747 ms.
RAM	4.0K	182.7K	182.7K
FLASH	-	141.9K	-
ACCURACY			36.90%

Unoptimized (float32)

Select

	IMAGE	CLASSIFIER	TOTAL
LATENCY	15 ms.	2,824 ms.	2,839 ms.
RAM	4.0K	721.6K	721.6K
FLASH	-	472.3K	-
ACCURACY			36.90%

Build

Image 6: Accuracy of camera hand gesture system.

Objectives

- System Integration - All

Integrate all subsystems using the LoRa, wifi, BLE, display, and servo into a single Smarthub. Making sure communication is still reliable.

- User Interface Development -Thiago

Finalize the user interface with real time data update, and make sure there is a smooth data display.

- Gesture recognition- All+Thiago mostly

Implement the camera- based hand gesture such as left swipe, right swipe, up, and down. We need to map to portrait and landscape as well.

- Completing BLE device detection- Anar

Improve BLE functionality to detect and interact with nearby devices, focusing on Android phones.

- Web interface and Remote Control- Johannes

Develop a web- based interface to allow users to monitor the system remotely.

- 3-D printing the screen stand- All

Design and 3D print a stable enclosure and mounting the servo to the screen

- Camera assembly-Anar

Soldering a wire connection between the camera and the esp32 dev.

- Power System- All

We need to identify the proper power solution for all components and make a final decision for our final system. Define how much power we will need.

- Testing for all combined systems.-All

Test Plan

- Camera Gesture Recognition- Thiago
 - Label all collected data samples.
 - Verify the accuracy of the gesture.
- LoRa Communication- Johannes, Anar
 - Complete verify the data transmission between the transmitter and receiver.
- BLE Communication-Anar
 - Scan and select BLE devices nearby. Aiming Android devices.
 - Verify the successful connection with BLE
 - Complete testing data transmission
- Display and UI testing- Thiago and Johannes
 - Verify the correct rendering of UI elements overall.
 - Ensure proper functionality in both landscape and portrait. Final check up after adding all elements.

- Servo Motor
 - Ensure stable operation with external power supply. Currently checked with classroom power supply.